PES UNIVERSITY

Department of Computer Science and Engineering

Software Engineering | UE23CS341A

SECURITY LOG

Vehicle Parking System

Security Design and Architecture — Unit 2, Lectures 2–3

Item	Details
Project repository	github.com/Sam-0960/Vehicle-Parking-System---SE---D-sec
Prepared for	Software Engineering group project
Purpose	To identify possible security risks, suggest safeguards and record what still needs to be checked.
Basis	Project documents and course notes on Security Design and Architecture.
Version	0.2 — revised plain-language draft

This document is a working record. The risks and controls below are proposals unless the team adds evidence showing that they have been implemented or tested.

1. Introduction and Security Objectives

1.1 Purpose and scope

This log describes security issues that could affect the Vehicle Parking System (VPS), a Software Engineering course project. It covers the application, the data it handles, how information moves between components, the deployment setup and any external services, such as a payment gateway.

The aim is to identify important data, consider how the system could be attacked, and list safeguards and tests for the team to review. The log should be updated as the project changes.

Some components and user roles mentioned here are assumptions because they still need to be checked against the project's SRS and design. No security tests or incidents are claimed as completed unless evidence is added.

1.2 Security objectives

The system should meet these basic security goals:

Confidentiality: Only authorised people should be able to see user profiles, vehicle numbers, booking history and payment-related information.

Integrity: Booking details, available slots, prices and payment records should only change through approved actions.

Replay protection: Reusing an old booking or payment request should not create a duplicate booking or charge.

Availability: Booking and entry/exit functions should remain usable during heavy traffic or repeated abusive requests.

Accountability: Important actions, such as logins, role changes, refunds and manual slot changes, should be recorded so they can be traced.

1.3 Status terms

Status	Meaning
Proposed	Suggested control; not confirmed as implemented.
Implemented	Present in code or configuration, with evidence such as a commit, pull request or screenshot.
Verified	Implemented and successfully checked, with test evidence recorded.

2. Assets and Potential Threats

2.1 Where important data is handled

Asset	Origin	In transit	At rest	In use
User credentials and profile	Registration/login page (assumed)	Client to server	User table	Login and profile view
Vehicle and booking data	Booking form or attendant desk (assumed)	Client to app; app to DB	Reservation table	Slot display and verification
Payment and billing records	Payment page/exit counter (assumed)	App to payment gateway (assumed)	Billing records; card data not planned	Receipts and reports
Slot inventory and prices	Admin dashboard (assumed)	Admin client to server	Slot and tariff tables	Availability and fee calculation
Admin accounts and audit logs	Admin setup/system events	App to log store	Log files or table	Security review

2.2 Trust boundaries

Customers should only access their own information. Attendants and administrators should be able to use only the functions allowed for their roles. Anonymous visitors, unauthenticated API callers and other customers must not be able to access protected data. A payment gateway response should be trusted only after it has been verified.

The architecture should show the boundaries between the browser and application server, application server and database, application server and payment gateway, and admin interface and application server.

2.3 Possible threats

Threat	Example
Unauthorised access	Guessing or stealing passwords, or opening pages without logging in.
SQL injection	Entering malicious text into login, search or vehicle-number fields to interfere with database queries.

Threat	Example
Privilege escalation	A customer accessing attendant or administrator functions by changing a URL or request.
Data leakage	Personal or vehicle data appearing in errors, logs, unencrypted traffic or overly broad API responses.
Booking manipulation	Changing another user's booking, changing the price or time, or booking one slot twice.
Payment attacks	Changing the amount, replaying a payment request, forging a gateway response or storing card details unsafely.
Session attacks	Using a stolen session token or replaying a captured request.
Service abuse	Sending many requests or creating unpaid bookings to hold slots.

3. Risk Assessment

Likelihood and impact are rated Low (1), Medium (2) or High (3). The risk score is their product: 1–2 is Low, 3–4 is Medium, 6 is High and 9 is Critical. These are initial estimates for the team to review, not measured results.

ID	Risk	Area affected	L	I	Level	Suggested mitigation
R-01	Weak or stolen credentials	Login/authentication	M	H	High	Strong passwords, salted password hashes, login throttling/lockout and MFA for admins.
R-02	SQL injection	Database, search and login	M	H	High	Parameterised queries, input validation and a database account with limited permissions.
R-03	Customer gains staff/admin access	Authorisation/admin area	M	H	High	Check permissions on the server for every request; deny access by default.
R-04	Personal or vehicle data leaks	API, logs, database, network	M	H	High	Use HTTPS/TLS, protect sensitive stored data, return only necessary fields and avoid detailed error messages.
R-05	Booking changes or double booking	Reservation and slot inventory	H	M	High	Check booking ownership, lock slots during transactions and calculate price/time on the server.
R-06	Payment tampering or replay	Payment integration	M	H	High	Use a hosted gateway/tokenisation, verify callbacks and prevent duplicate processing.
R-07	Session hijacking/request replay	Session handling	M	M	Medium	Use secure cookies, session expiry and rotation, and duplicate-request protection.
R-08	API abuse or slot hoarding	Booking endpoints	M	M	Medium	Rate-limit requests and expire temporary reservations.
R-09	Unsafe defaults or configuration	Deployment/admin settings	L	H	Medium	Disable debug mode, remove default credentials and keep secrets out of source code.
R-10	Staff/admin misuse	Admin area, database, logs	L	H	Medium	Limit permissions and record administrative actions.
R-11	Missing security logs	Logging and monitoring	M	M	Medium	Record authentication, payment and admin events; review unusual activity.

The components listed in the risk register are assumptions until they are checked against the actual project design.

4. Security Event Log

Record an entry when a security issue, failed test, suspicious activity, vulnerable dependency or configuration problem is found. Add the date and evidence. No security incident has been reported in this draft.

Log ID	Date	Event	Severity	Action/status	Evidence
SEC-001	14-Oct-2026	Created the initial security log and risk register from course notes.	Informational	Shared for team review; closed.	This document, v0.2
SEC-002	To be added	Check the risk register and component names against the SRS/design.	Informational	Open until the team confirms assumptions.	Commit or document reference
SEC-003	YYYY-MM-DD	Event description	Severity	Action taken; open/closed	Screenshot, log or issue link
SEC-004	YYYY-MM-DD	Event description	Severity	Action taken; open/closed	Screenshot, log or issue link
SEC-005	YYYY-MM-DD	Event description	Severity	Action taken; open/closed	Screenshot, log or issue link

5. Security Controls and Mitigation

The controls below are recommendations based on the course topics. Keep their status as Proposed until the team has evidence of implementation. Mark a control Verified only after its related test passes.

ID	Control	Risks	Status
C-01	Improve login security: password policy, bcrypt/Argon2 hashing, lockout, session expiry/rotation and admin MFA.	R-01, R-07	Proposed
C-02	Use role-based access control. Check every request on the server and separate customer, attendant and admin permissions.	R-03, R-05, R-10	Proposed
C-03	Validate inputs and use parameterised database queries instead of building SQL from strings.	R-02	Proposed
C-04	Use TLS for client and gateway traffic; protect sensitive stored data and use mTLS if services are separated.	R-04, R-06, R-07	Proposed

ID	Control	Risks	Status
C-05	Check booking ownership, use unpredictable IDs, lock slots within transactions and calculate fees on the server.	R-05	Proposed
C-06	Use a hosted payment gateway or tokenisation; do not store card numbers/CVV; verify callbacks and prevent duplicate processing.	R-06	Proposed
C-07	Rate-limit login, booking and payment APIs; expire temporary slot holds.	R-01, R-08	Proposed
C-08	Disable debug mode, remove default credentials, store secrets outside source code and expose only necessary ports.	R-09	Proposed
C-09	Log logins, role changes, refunds and slot overrides. Do not log passwords or payment data; restrict log access.	R-04, R-10, R-11	Proposed
C-10	Keep the database inaccessible directly from clients and use a database account with limited permissions.	R-02, R-04, R-10	Proposed

5.1 Architecture review

Question	Current status	Evidence to add
Are sensitive components separated?	Yes / Partial / No	Explain database, payment and admin separation.
Are APIs authenticated, rate-limited and validated?	Yes / Partial / No	List the endpoints reviewed.
Is encryption used in transit and at rest where needed?	Yes / Partial / No	Add TLS and storage configuration evidence.
Are default settings secure?	Yes / Partial / No	Add configuration review reference.

6. Security Testing Checklist

The tests below have not been run as part of this draft. Record a result and attach evidence only after carrying out each test. A secure result should match the expected behaviour listed.

ID	Test	Risk	Expected result	Result
T-01	Enter an incorrect password repeatedly.	R-01	Generic error; throttling or lockout works and attempts are logged.	Not executed
T-02	Open a protected page/API without logging in.	R-01	Redirect to login or return 401; no protected data is returned.	Not executed
T-03	Enter SQL-like input in login, search and plate fields.	R-02	Input is treated as text or rejected; no database error or data change.	Not executed
T-04	Use a customer account to open an admin/attendant URL or API.	R-03	Access is denied (403); attempt is logged where supported.	Not executed
T-05	Change a booking/user ID in a request to access someone else's booking.	R-04, R-05	Access is denied; another user's data is not returned.	Not executed
T-06	Change the amount or duration sent in a payment request.	R-06	Server recalculates the fee and rejects tampered values.	Not executed
T-07	Resend a booking or payment request.	R-06, R-07	Duplicate processing is prevented; no second booking or charge.	Not executed
T-08	Have two users try to book the same slot at once.	R-05	Only one succeeds; the other gets a clear conflict message.	Not executed
T-09	Try to use plain HTTP and inspect traffic.	R-04	Traffic uses TLS and HTTP redirects to HTTPS.	Not executed
T-10	Send malformed input to trigger an error.	R-04, R-09	Generic error; no stack trace or SQL details are exposed.	Not executed
T-11	Check stored credentials and payment data.	R-04, R-06	Passwords are hashed; card number and CVV are not stored.	Not executed
T-12	Log out, then reuse the old session token; inspect cookie flags.	R-07	Old token is invalid; cookies use Secure and HttpOnly where applicable.	Not executed
T-13	Send many requests to login and booking APIs.	R-08	Rate limit is applied and the service remains responsive.	Not executed
T-14	Review logs after the above tests.	R-11, R-04	Relevant authentication/admin/payment events appear; secrets are absent.	Not executed
T-15	Check for default accounts and debug mode.	R-09	No default credentials; debug mode is disabled in deployment.	Not executed

7. Incident Response and Conclusion

7.1 Severity levels

Severity	Example	Suggested response
Critical	Confirmed data breach, payment fraud or admin account takeover.	Contain immediately and notify the faculty/stakeholders.
High	Successful SQL injection or privilege escalation.	Contain the issue and begin fixing it the same day.
Medium	Repeated failed logins, rate-limit bypass or configuration issue.	Plan and complete the fix in the current sprint.
Low	Minor information disclosure or a missing security header.	Add the fix to the backlog.

7.2 Response steps

1. **Detect and report:** Record the event, date and evidence in the Security Event Log.

2. **Assess:** Decide the severity and identify the affected component and data.

3. **Contain:** Disable the affected account, endpoint or feature if needed. Revoke sessions or keys and preserve logs/screenshots.

4. **Fix:** Correct the code or configuration issue and raise a change request for any design gap.

5. **Recover:** Restore normal service and rerun the relevant test.

6. **Review:** Record lessons learned and update the risk register and threat map.

7.3 Roles

Role	Responsibility	Assigned to
Incident lead	Coordinates the response and decisions.	[Name]
Technical owner	Investigates and applies the fix.	[Name]
Documentation/log keeper	Maintains the log and evidence.	[Name]
Faculty contact	Informed about High and Critical events.	[Name]

7.4 Conclusion

This log gives the Vehicle Parking System team a starting point for reviewing security. It lists the main assets, trust boundaries and likely threats, along with suggested controls and tests. At the time of writing, the controls are still marked Proposed, the tests have not been executed, and no incidents have been recorded.

The next steps are to check the assumptions against the SRS and design documents, prioritise the high-rated risks (R-01 to R-06), implement the agreed controls, attach evidence and update the checklist as tests are completed.

Reference: Software Engineering, Unit 2 Lecture 2–3, Security Design and Architecture, Department of CSE, PES University (UE23CS341A).